

```

        <li>There are no entries for this topic yet.</li>
    {% endfor %}
</ul>

{% endblock content %}

```

We extend *base.html*, as we'll do for all pages in the project. Next, we show the text attribute of the topic that's been requested ❶. The variable `topic` is available because it's included in the context dictionary. We then start a bulleted list ❷ to show each of the entries and loop through them ❸, as we did with the topics earlier.

Each bullet lists two pieces of information: the timestamp and the full text of each entry. For the timestamp ❹, we display the value of the attribute `date_added`. In Django templates, a vertical line (`|`) represents a template *filter*—a function that modifies the value in a template variable during the rendering process. The filter `date:'M d, Y H:i'` displays timestamps in the format *January 1, 2022 23:00*. The next line displays the value of the current entry's text attribute. The filter `linebreaks` ❺ ensures that long text entries include line breaks in a format understood by browsers, rather than showing a block of uninterrupted text. We again use the `{% empty %}` template tag ❻ to print a message informing the user that no entries have been made.

Links from the Topics Page

Before we look at the topic page in a browser, we need to modify the topics template so each topic links to the appropriate page. Here's the change you need to make to *topics.html*:

```

topics.html  --snip--
             {% for topic in topics %}
               <li>
                 <a href="{% url 'learning_logs:topic' topic.id %}">
                   {{ topic.text }}</a></li>
               </li>
             {% empty %}
             --snip--

```

We use the URL template tag to generate the proper link, based on the URL pattern in `learning_logs` with the name `'topic'`. This URL pattern requires a `topic_id` argument, so we add the attribute `topic.id` to the URL template tag. Now each topic in the list of topics is a link to a topic page, such as `http://localhost:8000/topics/1/`.

When you refresh the topics page and click a topic, you should see a page that looks like Figure 18-5.

NOTE

There's a subtle but important difference between `topic.id` and `topic_id`. The expression `topic.id` examines a topic and retrieves the value of the corresponding ID. The variable `topic_id` is a reference to that ID in the code. If you run into errors when working with IDs, make sure you're using these expressions in the appropriate ways.